Lab 9: Creating and Using Lookups in Splunk

Lab Overview

In this lab, you will learn to enrich your Splunk data using lookup tables. You'll create lookup files that add business context to raw event data, configure automatic lookups to seamlessly integrate this information, and use lookup commands to enhance your reports with meaningful product names and pricing information.

Learning Objectives

By the end of this lab, you will be able to:

- Create and upload lookup table files
- Configure lookup definitions
- Use the `inputlookup` command to verify lookup data
- Apply the `lookup` command to enrich search results
- Create automatic lookups for seamless data enrichment
- Add product names and prices to web application data
- Build revenue reports using enriched data
- Manage lookup permissions and sharing

Prerequisites

- Completed Labs 1-7
- Understanding of Splunk search commands
- Familiarity with CSV file format
- Access to Sales Dashboard from previous labs
- Basic knowledge of field mapping concepts

> **Data window**: The course dataset covers roughly **26 days** ending at the moment `labs/data/generate_course_data.py` was last run — deliberately sized to sit inside Splunk's **Last 30 days** preset. Set the time picker to **Last 30 days** for every exercise unless a step says otherwise.
>
> Regenerate the data within **4 days** of delivering the class; past that the preset starts clipping the oldest events and the answer-key counts drift.

Lab Environment Setup

Step 1: Verify Sample Data

Ensure required data is loaded:

```
index=main earliest=-30d latest=now
| stats count by sourcetype
```

Required sourcetypes:

- `access_combined_wcookie` - Web application logs with productId field
- `db_audit` - Database audit logs
- `linux_secure` - Linux security logs

Step 2: Prepare Lookup File

The lab includes a products.csv file in the course data folder with product information.

Key Concepts

What are Lookups?

Lookups add fields from external data sources to your events:

- Enrich events with reference data
- Add business context to technical data
- Map codes to descriptions
- Provide pricing and inventory information

Types of Lookups

Type	Description	Use Case
CSV Lookups	File-based lookups using CSV files	Product catalogs, user directories
KV Store Lookups	Database-backed lookups	Large datasets, frequently updated data
External Lookups	Script-based lookups	Dynamic data from APIs
Geospatial Lookups	Location-based lookups	IP geolocation, store locations

Lookup Workflow

1. **Upload File**: Add CSV file to Splunk
2. **Create Definition**: Define how to use the file
3. **Configure Mapping**: Map input/output fields

4. **Apply Lookup**: Manual or automatic application

Exercise 1: Create Product Lookup File

Scenario: The web application data contains only product IDs. Users want to see product names and prices in reports.

Task 1.1: Locate Products CSV File

The `products.csv` file is already provided in your course data folder with the following content:

```
productId,product_name,categoryId,price,Code
DB-SG-G01,Mediocre Kingdoms,STRATEGY,24.99,A
DC-SG-G02,Dream Crusher,STRATEGY,39.99,B
FS-SG-G03,Final Sequel,STRATEGY,24.99,C
WC-SH-G04,World of Cheese,SHOOTER,24.99,D
WC-SH-T02,World of Cheese Tee,TEE,9.99,E
PZ-SG-G05,Puppies vs. Zombies,STRATEGY,4.99,F
CU-PG-G06,Curling 2014,SPORTS,19.99,G
MB-AG-G07,Manganiello Bros.,ARCADE,39.99,H
MB-AG-T01,Manganiello Bros. Tee,TEE,9.99,I
FI-AG-G08,Orvil the Wolverine,ARCADE,39.99,J
BS-AG-G09,Benign Space Debris,ARCADE,24.99,K
SC-MG-G10,SIM Cubicle,SIMULATION,19.99,L
WC-SH-A01,Holy Blade of Gouda,ACCESSORIES,5.99,M
WC-SH-A02,Fire Resistance Suit of Provolone,ACCESSORIES,3.99,N
GT-SC-G01,Grand Theft Scooter,SHOOTER,24.99,O
WSC-MG-G10,SIM Cubicle Tee,TEE,9.99,P
```

Task 1.2: Upload Lookup File

1. Navigate to **Settings** → **Lookups** → **Lookup table files**
2. Click **New Lookup Table File**
3. Configure:
   - **Destination app**: search
   - **Upload a lookup file**: Browse to your course data folder and select `products.csv`
   - **Destination filename**: products.csv
4. Click **Save**

✓ **Checkpoint**: File uploaded successfully

Exercise 2: Configure Lookup Definition

Task 2.1: Create Lookup Definition

1. Navigate to **Settings** → **Lookups** → **Lookup definitions**
2. Ensure **Search & Reporting** is selected for App context
3. Click **New Lookup Definition**
4. Configure:
   - **Destination app**: search
   - **Name**: `products_lookup`
   - **Type**: File-based
   - **Lookup file**: products.csv
5. Click **Save**

Task 2.2: Verify Lookup Definition

Test the lookup using `inputlookup`:

```
| inputlookup products_lookup
```

Expected Results: Table showing all products with columns:

- productId
- product_name
- price
- categoryId

✓ **Checkpoint**: All products displayed with correct fields

Exercise 3: Use Lookup in Searches

Task 3.1: Manual Lookup Application

Search for successful purchases:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
```

Task 3.2: Apply Lookup Command

Add product information using lookup:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
| lookup products_lookup
    productId as productId
    OUTPUT product_name as ProductName
```

Observation: New field `ProductName` appears in field list

Task 3.3: Create Product Sales Report

Generate sales count by product name:

```
index=main sourcetype=access_combined_wcookie
  status=200
  file=success.do
| lookup products_lookup
    productId as productId
    OUTPUT product_name as ProductName
| stats count by ProductName
```

Expected Results: Table showing product names with sales counts

✓ **Checkpoint**: Products show readable names instead of IDs

---

Exercise 4: Configure Automatic Lookup

Task 4.1: Create Automatic Lookup

1. Navigate to **Settings** → **Lookups** → **Automatic lookups**
2. Click **New Automatic Lookup**
3. Configure:
   - **Destination app**: search
   - **Name**: `products_auto_lookup`
   - **Lookup table**: products_lookup
   - **Apply to**: sourcetype
   - **named**: `access_combined_wcookie`

- **Lookup input fields**:
  - `productId = productId`
- **Lookup output fields**:
  - `product_name = ProductName`
  - `price = Price`

4. Click **Save**

Task 4.2: Verify Automatic Lookup

Test that fields are automatically added:

```
index=main sourcetype=access_combined_wcookie
  file=success.do
  status=200
| table productId, ProductName, Price
```

Expected Results: ProductName and Price fields automatically populated

✓ **Checkpoint**: No lookup command needed; fields appear automatically

---

Exercise 5: Revenue Analysis

Task 5.1: Calculate Revenue by Product

Using the automatic lookup, calculate total revenue:

```
index=main sourcetype=access_combined_wcookie
  file=success.do
  status=200
| stats sum(Price) as Revenue by ProductName
```

Task 5.2: Find Top Revenue Product

Sort to find highest revenue generator:

```
index=main sourcetype=access_combined_wcookie
  file=success.do
  status=200
| stats sum(Price) as Revenue by ProductName
| sort -Revenue
```

✓ **Checkpoint**: Record top revenue product: _____

Task 5.3: Add to Dashboard

1. Click **Save As** → **Dashboard Panel**
2. Configure:
   - **Dashboard**: Existing
   - **Dashboard Title**: Sales Dashboard
   - **Panel Title**: `Revenue by Product`
3. Click **Save** → **View Dashboard**

Challenge Exercises

Challenge 1: Customer Category Analysis

Create a lookup that maps customer IPs to customer categories:

1. Create customer_categories.csv:

```
clientip,customer_type,region
87.194.216.51,Premium,Europe
192.168.1.100,Standard,US-East
10.0.0.50,Premium,US-West
```

2. Configure lookup and apply to find:
   - Revenue by customer type
   - Top products by region

Challenge 2: Time-Based Pricing

Implement dynamic pricing lookup:

1. Create seasonal_pricing.csv with date ranges
2. Use time-based lookup matching
3. Calculate revenue with seasonal adjustments

Challenge 3: Multi-Field Lookup

Create complex lookup matching multiple fields:

```
| lookup store_inventory
    productId as productId
    store_location as location
    OUTPUT stock_level, reorder_point
```

Advanced Lookup Techniques

Lookup Command Options

Option	Purpose	Example
local=true	Don't share across search heads	`lookup local=true ...`
update=true	Overwrite existing fields	`lookup update=true ...`

Case Sensitivity

Configure case-insensitive lookups:

1. Edit lookup definition
2. Advanced options → Case sensitive match: No

Default Values

Set default values for missing matches:

1. Edit lookup definition
2. Advanced options → Default matches

Lookup Acceleration

For large lookup files:

1. Consider KV Store lookups
2. Enable lookup acceleration
3. Use indexed lookups for performance

Troubleshooting Guide

Common Issues and Solutions

Issue	Solution
Lookup file not found	Check file upload and permissions
Fields not matching	Verify field names are exact (case-sensitive)
No results from inputlookup	Check lookup definition configuration
Automatic lookup not working	Verify sourcetype spelling and field mapping
Price showing as string	Ensure numeric fields are properly formatted

Debugging Lookups

Test lookup configuration:

```
| inputlookup products_lookup
| stats count by productId
```

Verify field extraction:

```
index=main sourcetype=access_combined_wcookie
| head 10
| table productId
```

Check automatic lookup:

```
index=main sourcetype=access_combined_wcookie
| head 10
| fieldsummary
```

Best Practices

Lookup File Management

1. **Naming Convention**: Use descriptive names (products_lookup, not lookup1)
2. **Version Control**: Keep backup copies of lookup files
3. **Documentation**: Document field meanings and update frequency
4. **Size Limits**: Keep CSV files under 50MB for performance

Field Mapping

1. **Consistent Names**: Use standard field naming conventions
2. **Data Types**: Ensure numeric fields are numbers, not strings
3. **Missing Values**: Handle nulls appropriately
4. **Multi-value Fields**: Consider using mvexpand for multi-value lookups

Performance Optimization

1. **Automatic vs Manual**: Use automatic for frequently used lookups
2. **Local vs Shared**: Use local=true for search-head specific data
3. **Acceleration**: Enable for large, frequently accessed lookups
4. **KV Store**: Consider for lookups needing frequent updates

---

Cleanup

To clean up after this lab:

1. **Remove Automatic Lookup**:
   - Settings → Lookups → Automatic lookups
   - Delete `products_auto_lookup`
2. **Remove Lookup Definition**:
   - Settings → Lookups → Lookup definitions
   - Delete `products_lookup`
3. **Remove Lookup File**:
   - Settings → Lookups → Lookup table files
   - Delete `products.csv`

Key Takeaways

✅ **You've learned to:**

- Upload and manage lookup table files
- Create lookup definitions for file access
- Use inputlookup to verify lookup data
- Apply lookups manually with the lookup command
- Configure automatic lookups for seamless enrichment
- Calculate business metrics using enriched data

- Add lookups to dashboards for business reporting

Next Steps

- Explore KV Store lookups for dynamic data
- Learn about external lookups for API integration
- Study geospatial lookups for location intelligence
- Practice time-based lookups for temporal data

Quick Reference

Essential Lookup Commands

Command	Purpose	Example
`inputlookup`	View lookup contents	`| inputlookup products_lookup`
`lookup`	Apply lookup to results	`| lookup products_lookup productId OUTPUT product_name`
`outputlookup`	Write results to lookup	`| outputlookup new_products.csv`
`rest`	List available lookup files	`| rest /services/data/lookup-table-files`

Lookup Command Syntax

```
| lookup [local=<bool>] [update=<bool>]
    <lookup_name>
    <input_field> [AS <event_field>]
    [OUTPUT | OUTPUTNEW <output_field> [AS <new_field>]]
```

Common Lookup Patterns

Pattern	Use Case	Example
Simple enrichment	Add product names	`lookup products_lookup productId OUTPUT product_name`
Multiple outputs	Add name and price	`lookup products_lookup productId OUTPUT product_name, price`
Field renaming	Custom field names	`lookup products_lookup productId OUTPUT product_name AS ProductName`
Update existing	Overwrite fields	`lookup update=true products productId OUTPUT price`

Answer Key

🔑 **Instructor answer key**

Exercise 2

- Lookup definition name: products_lookup
- Lookup file: products.csv

Exercise 4

- Automatic lookup name: products_auto_lookup
- Applied to sourcetype: access_combined_wcookie

Exercise 5

- Top revenue product: Orvil the Wolverine ($7,998.00)
- Revenue calculation field: Price

Product List

- Dream Crusher: $39.99
- Benign Space Debris: $24.99
- Curling 2014: $19.99
- Final Sequel: $24.99
- Fire Resistance Suit of Provolone: $3.99